

Loops(for , while ,do...while)



CONTENT

01

Introduction

02

for loop

03

While loop

04

do...while loop

05

Nested Loops and Loop
Control in JavaScript





01

Introduction

Introduction

**A loop allows you to execute a block of code multiple times —
as long as a specific condition is true**





02

for loop

for loop

Used when you know how many times to repeat.

Syntax:

```
for (initialization; condition; increment) {  
    // code to run  
}
```

Flow

- Initialize
- Check condition
- Run the block
- Increment
- Repeat until condition is false



Examples

JS index.js X

JS index.js > ...

```
1  for(let i=1;i<=5;i++){  
2    console.log(i);  
3  
4  }
```

Output

[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"

1
2
3
4
5

[Done] exited with code=0 in 0.078 seconds





03

while loop

while loop

Used when you don't know how many times you need to loop; the loop continues as long as the condition is true.

Syntax:

```
while (condition) {  
    // code to run  
}
```

Note

Be careful with the condition or it may run forever!



Examples

```
JS index.js ×  
JS index.js > ...  
1  let i = 1;  
2  while (i <= 5) {  
3      console.log("Number:", i);  
4      i++;  
5  }
```

Output

```
[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"
```

```
Number: 1
```

```
Number: 2
```

```
Number: 3
```

```
Number: 4
```

```
Number: 5
```

```
[Done] exited with code=0 in 0.081 seconds
```





04

do...while loop

do...while loop

It is guaranteed to run at least once, even if the condition is false.



Syntax:

```
do {  
    // code to run  
} while (condition);
```

Examples

```
JS index.js ×  
JS index.js > ...  
1 let i = 1;  
2 do {  
3   console.log("Number:", i);  
4   i++;  
5 } while (i <= 5);
```

Output

```
[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"
```

```
Number: 1
```

```
Number: 2
```

```
Number: 3
```

```
Number: 4
```

```
Number: 5
```

```
[Done] exited with code=0 in 0.081 seconds
```





05

Nested Loops and Loop Control in JavaScript

Nested Loops

A loop inside another loop is called a nested loop. It helps when you need to work with multi-level data — like rows and columns, grids, or matrices.

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Example:

```
for(){           //outer loop  
  
    for(){       //inner loop  
  
    }  
}
```



Loop Control Statement

break Statement

The break statement **immediately exits the loop.**

JS index.js ×

JS index.js > ...

```
1  for (let i = 1; i <= 5; i++) {  
2    if (i === 3) {  
3      break;  
4    }  
5    console.log(i);  
6  }
```

Output

[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"

1
2

[Done] exited with code=0 in 0.136 seconds

When i === 3, the loop stops completely.



Loop Control Statement

continue Statement

The continue statement skips the current iteration and moves to the next.

JS index.js ×

JS index.js > ...

```
1  for (let i = 1; i <= 5; i++) {  
2    if (i === 3) {  
3      continue;  
4    }  
5    console.log(i);  
6  }
```

Output

[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"

1
2
4
5

[Done] exited with code=0 in 0.079 seconds

When `i === 3`, the loop stops completely.



Summary

Loop Type	When to Use	Condition Checked	Runs at Least Once
for	Fixed number of iterations	Before loop	No
while	Unknown number of times, pre-condition	Before loop	No
do...while	Unknown times but must run once	After loop	✓ Yes

Control Keyword	Description	Effect
break	Exits the current loop entirely	Stops the loop
continue	Skips current iteration	Goes to next loop iteration

